

ESENCIALES DE POO

Nacho Garbayo, 2022-2023

ALMACENAMIENTO

POO

MEMORIA

PILA (STACK): acceso rápido del programa (ordenado). Variables con tamaño ya definido.

- Variables locales de un método durante su ejecución y todo lo asociado a sus datos primitivos.
- Código de los métodos
- Referencias a objetos creados

MONTÓN (HEAP): acceso lento con búsqueda necesaria.

- Objetos creados, cuya referencia está en la pila
- Lo gestiona el recolector de basura
- String directas sin repetición en el String Pool; Strings indirectas con posible repetición en el montón a secas

ALIASING Y REFERENCIAS

POO

Principios

Todo nombre de un objeto es una referencia a una posición de memoria.

El operador NEW hace la reserva de memoria.

Aliasing

El aliasing se produce al igualar objetos, igualando en realidad sus referencias. Evita de algún modo la encapsulación de las clases, pues a través de las referencias se podrían modificar atributos privados desde métodos que no pertenecen a la clase considerada. Tiene como consecuencia programas más difíciles de mantener desde la perspectiva de la Programación Orientada a Objetos.

Clase STRING

Una instancia de tipo STRING es inmutable. Al crearse se reserva memoria para su referencia y esta apunta a la cadena de texto deseada.

Si se le asigna otro valor, la referencia pasa a ser a otra cadena de texto, no cambiando la cadena original

Se crea:

- **Directamente** `x = "cadena"`: se crea la referencia y se crea la cadena en el String Pool del montón. Si se repite la cadena en el STRPOOL, esta solo se crea una vez.
- **Indirectamente** (por construcción) `x = new String("cadena")`: se crea la referencia y la cadena en el montón fuera del STRPOOL, tantas veces como se repita.

AUTOBOXING, UNBOXING: WRAPPERS

POO

Autoboxing: empaqueta, tipo de dato primitivo a Wrapper. Tiene lugar:

- Un método con argumento Wrapper recibe un primitivo
- A un Wrapper se le asigna un primitivo

Unboxing: desempaqueta: Wrapper a tipo de dato primitivo. Sucede:

- Un método con argumento primitivo recibe un Wrapper
- A un primitivo se le asigna un wrapper


```
public static void main(String[] args) {  
    Integer aa= 10;  
    Integer bb= 20;  
    if(aa>bb)  
        System.out.println("aa + bb = " + suma(aa, bb));  
}  
  
public static int suma(int a, int b) {  
    return a+b;  
}
```

autoboxing

unboxing

```
public static void main(String[] args) {  
    int aa= new Integer(10);  
    int bb= new Integer(20);  
    if(aa>bb)  
        System.out.println("aa + bb = " + suma(aa, bb));  
}  
  
public static int suma(Integer a, Integer b) {  
    return a+b;  
}
```

unboxing

autoboxing

EQUALS

Se utilizan primitivos (no hay referencias), wrappers (inmutables como Strings) o Strings (inmutables)

```
@Override
public boolean equals (Object obj) {
    if (this == obj)
        return true;
    if (obj == null)
        return false;
    if (this.getClass() != obj.getClass())
        return false;

    final CLASE other = (CLASE) obj;
    if (this.Criterio != other.Criterio)
        return false;
    return true;
}
```

CONJUNTOS DE DATOS

Collection, List, ArrayList, Set, Map, HashMap, Iterator

COMPARATIVA

- 0 Arrays Elementos accesibles a través de un índice
- 0 Colección Grupo de objetos
<https://docs.oracle.com/javase/8/docs/api/java/util/Collection.html>
- 0 Iterador Objeto que itera sobre una colección
<https://docs.oracle.com/javase/8/docs/api/java/util/Iterator.html>
- 0 Lista Colección ordenada de objetos
<https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html>
- 0 Conjunto Colección sin objetos duplicados
<https://docs.oracle.com/javase/8/docs/api/java/util/Set.html>
- 0 Mapa Objeto que relaciona claves con valores
<https://docs.oracle.com/javase/8/docs/api/java/util/HashMap.html>

4 grandes Tipos

- **Array**: se puede construir de primitivos (única estructura que lo permite), wrappers u objetos. Tamaño fijo y posiciones contiguas en memoria.
- **Collection**: de ella heredan List, Set, HashSet, ArrayList. NO se recorre a través de un índice, no tiene orden. Métodos: add, contains, isEmpty, remove, iterator. contains y remove usan equals. Se puede recorrer con for-each, pero no se puede modificar durante el recorrido. Un for-each se interpreta en compilación como un iterator.

- **Map:** de ella hereda HashMap. Asocia cada valor con una clave, luego las claves no se pueden repetir, pero sí los objetos. Utiliza equals, por lo que la clave también debería ser inmutable. Métodos: get, put, containsValue, containsKey, remove, values (collection), keySet (set), entrySet (set MapEntry<K,V>).
- **Iterator:** recorre los elementos de una Collection. Su puntero señala al siguiente elemento. Métodos: hasNext, next, remove. Permite la eliminación de elementos durante el recorrido de una colección. Solo se puede recorrer una vez, no puede volver atrás.

```
Iterator<Clase> it = Clase.iterator();  
while (it.hasNext()) {  
    Clase c = it.next();  
    // Tratar c  
}
```

Subtipos

- **List**: de él hereda ArrayList. Se puede recorrer usando un índice, tiene orden. Admite duplicados. Métodos: get, set, remove.
- **ArrayList**: al invocar a NEW se reservan 10 elementos y se redimensiona de forma dinámica. Si se recorre con el índice, se puede modificar durante el recorrido. Array en esteroides. El índice cuenta desde 0, ..., n-1.
- **Set**: de él hereda HashSet. NO ADMITE REPETICIÓN DE SUS DATOS. Solo puede haber un null. Se gestiona con equals.
- **HashSet**: el orden de los datos se sigue según el hash code: acceso $O(1)$.
- **HashMap**: la igualdad del equals se complementa mediante el hash code. Primero se compara hash code entre 2 elementos, luego equals.

Las listas ocupan menos memoria que los mapas.

Los mapas son más rápidos que las listas.

Collection	Ordering	Random Access	Key-Value	Duplicate Elements	Null Element
ArrayList	✓	✓	✗	✓	✓
LinkedList	✓	✗	✗	✓	✓
HashSet	✗	✗	✗	✗	✓
TreeSet	✓	✗	✗	✗	✗
HashMap	✗	✓	✓	✗	✓
TreeMap	✓	✓	✓	✗	✗

HERENCIA, POLIMORFISMO, INTERFACES

POO

Generales

- **super** permite acceder desde la clase derivada a los atributos, métodos y constructores de la clase inmediatamente base sobre los que existe cierto nivel de visibilidad (super.Método() o super(argumentos) si es constructor)
- Se pueden sobrescribir métodos con @Override
- Las clases abstractas no se pueden instanciar. Si tiene al menos un método abstracto

```
public abstract <tipo_dato> nombre_método(<tipo_dato> nombre *);
```

debe ser abstracta.

- Una clase final debería ocupar las hojas de la jerarquía de clases.

- Una clase abstracta...
 - Puede tener todos los métodos abstractos
Debería tener tantos métodos implementados como sea posible
 - Puede tener todos los métodos implementados
 - Puede ocupar cualquier lugar de la jerarquía de clases
Debería ocupar los niveles superiores de la jerarquía de clases
 - Puede ser una clase derivada de una clase no abstracta
Su clase base debería ser una clase abstracta
 - Puede no tener constructores
Debería tener tantos constructores como sea necesario
 - Puede tener atributos de cualquier tipo

UPCASTING, DOWNCASTING (POLIMORFISMO)

Las clases abstractas no solamente se usan para estructurar las jerarquías de clases, sino que también para restringir los métodos que pueden invocar los objetos.

Sí que se pueden crear entidades de clases abstractas, pero no aplicar NEW sobre ellas:

```
CartaDeEquipamiento cartaEquip= infEuropa;
```



```
CartaDeEquipamiento cartaEquip= new CartaDeEquipamiento();
```



Si Infantería ES UNA carta de equipamiento, entonces:

```
CartaDeEquipamiento carta = new CartaDeEquipamiento();  
Infanteria inf = new Infanteria();  
    // Upcasting: asignar a una direcc. de clase madre instancia de clase hija  
CartaDeEquipamiento carta = inf;  
    // Downcasting: asignar a una dir. de clase hija instancia de clase madre  
Infanteria inf = (Infanteria) carta;
```

No implican nuevas reservas de memoria y el DOWNCASTING no siempre es posible por la compatibilidad de los atributos y métodos de las clases. Después de la asignación, el objeto solo puede acceder a los métodos de la nueva clase (a la izquierda).

MECANISMOS EN LOS MÉTODOS

En UPCASTING, si un método está sobrescrito en ambas clases, se utiliza la implementación del método de la clase hija, aunque se comporte como la clase madre (si esto no pasase, no habría polimorfismo con clases abstractas como madres).

UPCASTING nunca produce error. DOWNCASTING puede generar error por la incompatibilidad de métodos y atributos.

Antes de utilizar un método con DOWNCASTING, se debe utilizar instanceof

< instancia instanceof Clase >

para comprobar si es posible (no evita los errores aun así, ya que el downcasting es unsafe).

MÉTODOS ESTÁTICOS Y POR DEFECTO

Una **interfaz** es la plantilla de una clase, con métodos. Una clase IMPLEMENTA una interfaz, mientras que en herencia, una clase EXTIENDE a otra (Herencia: “es una”; Composición: “tiene los atributos de”), o una interfaz EXTIENDE a otra.

Los atributos de una interfaz son IMPLÍCITAMENTE `public static final` (ctes.). Los métodos son IMPLÍCITAMENTE `public abstract`.

Clases abstractas	Interfaces
Se definen cuando las clases derivadas tienen algunos métodos comunes	Se definen cuando las clases que los implementan tienen diferentes implementaciones para diferentes objetos
La herencia múltiple no es posible	Se puede entender que la herencia múltiple es posible, aunque solo puede haber una clase base
Pueden tener métodos abstractos y métodos concretos	Solamente tienen métodos abstractos, además de métodos estáticos y métodos por defecto
Los métodos abstractos pueden ser públicos y protegidos	Los métodos abstractos deben de ser públicos
Tienen constructores	No tienen constructores
Pueden tener cualquier tipo de atributo	Los únicos atributos que puede tener son de tipo estático y final (son constantes)

MÉTODOS ESTÁTICOS Y POR DEFECTO

Son añadidos de Java al concepto de INTERFAZ.

- **DEFECTO**: se declaran e implementan en el interfaz, se heredarán por todas las clases que implementan el interfaz. Solo utiliza los argumentos del método por defecto y las funciones definidas en la interfaz. Pueden ser sobrescritos. (se denotan por “default public método...”)
- **ESTÁTICOS**: pueden ser invocados desde el inicio del programa sin crear ninguna instancia de la clase, con Clase.método. Se pueden definir en cualquier clase. No se heredan. Para su construcción, únicamente se pueden invocar métodos de la clase o del interfaz que sean estáticos, puesto que tienen que estar disponibles desde el arranque del programa

Métodos por defecto	Métodos estáticos
Solamente pueden estar definidos en los interfaces	Se pueden definir en interfaces y en clases
Pueden invocar cualquier tipo de método, incluyendo métodos por defecto, estáticos y abstractos	Solamente pueden invocar métodos que, a su vez, sean métodos estáticos
Son heredados por las clases y por los interfaces derivados del interfaz que los contiene	No pueden ser heredados, siendo métodos que son específicos de las clases o de los interfaces en los que están implementados
No están disponibles en el arranque del programa, es necesario crear un objeto para invocarlos	Están disponibles al arrancar el programa

EXCEPCIONES

POO

THROW EN MÉTODOS

Se deben separar detección y tratamiento de errores.

En un método se lanza la excepción

```
if(<condición>) {  
    <tipo_excepción> <nombre_objeto> = new <constructor>;  
    throw <nombre_objeto>;  
}
```

- **Restricciones entre los bloques try-catch**

- Si existe un bloque catch deberá existir necesariamente **un único bloque try**
- Un bloque try deberá tener asociado, **al menos**, un bloque catch
- Un bloque try **podría tener tantos** bloques catch **como** el número de tipos de excepciones se puedan lanzar en el bloque try
- Un bloque try **podría tener menos** bloques catch **que** el número de excepciones que se hayan lanzado en el bloque try
 - Si las excepciones que se lanzan tienen la misma clase base, entonces puede definirse **un único bloque catch**
 - Si el tratamiento es el mismo puede usarse **multi-catch**

249

También se puede usar multi-catch:

```
catch(<tipo_excepción> | <tipo_excepción> | * <nombre_excepción>)
```

PRIORIDAD DE EJECUCIÓN

- **Error** de compilación: si el argumento del bloque catch es una de las clases superiores a una clase A, después de ese catch no podrá existir un catch con la clase A, puesto que la excepción se habrá capturado en el catch de la clase superior.
- El bloque **finally** se ejecuta siempre; independientemente de la excepción que se haya capturado o incluso aunque no se haya capturado ninguna excepción. Tiene que haber al menos un bloque catch para definir finally.

Nota: `System.out.println("c");` imprime `"\nc"`, con salto de línea anterior, y `System.out.print("c");` imprime únicamente `"c"`.